
Implementation Document

for

TrackMate

Version 1.0

Prepared by

Group 4:

Group Name: Earth's Mightiest Dozens -2

Krishna Agrawal
Chandradip Karmakar
Nideesh H
Abubakar Siddique
Ankit Kumar Jha
Aman Gupta
Rushali Vijayakumar Myageri
Ayush Mangal
Sahil Agarwal
Aditi

240563
240296
240690
240043
240137
240100
240894
240242
240902
240048

krishnaa24@iitk.ac.in
chandradip24@iitk.ac.in
nideesh24@iitk.ac.in
abubakars24@iitk.ac.in
ankitkj24@iitk.ac.in
amanu24@iitk.ac.in
rushali24@iitk.ac.in
ayushn24@iitk.ac.in
sahilag24@iitk.ac.in
aditij24@iitk.ac.in

Course: CS253

Mentor TA: George TL

Date: 27/03/2026

TABLE OF CONTENTS

CONTENTS.....	2
REVISIONS.....	3
1 IMPLEMENTATION DETAILS.....	
2 CODEBASE.....	
3 COMPLETENESS.....	
APPENDIX A - GROUP LOG.....	

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft Type and Number	Full Name	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Implementation Details

This section outlines the programming languages, frameworks, libraries, database systems, and tools utilized in the development of the software, alongside the rationale for their selection.

1.1 Programming Languages

- **Dart (Frontend):** Used as the primary language for the client-side application.
 - Justification: Dart is specifically optimized for UI creation. Its Ahead-of-Time (AOT) compilation ensures high native performance on mobile devices, making it superior to JavaScript-based alternatives for smooth animations and transitions.
- **Python 3 (Backend):** Used for developing the server-side API and core business logic.
 - Justification: Python offers rapid development capabilities and an extensive ecosystem for backend engineering. With the adoption of native `asyncio`, it matches the concurrency performance of Node.js while maintaining cleaner, more readable syntax.

1.2 Frameworks & Core Libraries

- **Flutter (Frontend Framework):** A UI toolkit for building natively compiled applications.
 - Justification: Flutter allows for a single codebase to deploy across Android and iOS without sacrificing performance. Unlike WebView-based frameworks (like Ionic) or bridge-reliant frameworks (like React Native), Flutter renders its own UI components directly via the Skia/Impeller engine, providing a highly consistent and performant user experience.

- **FastAPI (Backend API Framework):** A modern, fast web framework for building APIs with Python.
 - Justification: FastAPI was chosen over older frameworks like Django or Flask due to its exceptional performance (built on Starlette and ASGI). It enforces static type hinting using **Pydantic**, which drastically reduces runtime bugs and automatically generates interactive API documentation (Swagger UI), saving significant developer time.
- **WebSockets (websockets):** Used for real-time, bi-directional communication between the client and server.
 - Justification: Essential for live data updates where traditional HTTP polling would create unnecessary server load and latency.

1.3 Database Systems & Data Management

- **PostgreSQL (Relational Database):** Inferred primary database, interfaced via **asyncpg** for asynchronous operations and **psycopg2-binary** for synchronous fallbacks.
 - Justification: PostgreSQL is an industry-standard, highly compliant relational database. Using **asyncpg** allows the FastAPI backend to handle high-throughput database queries concurrently without blocking the main event loop. (Note: **aiosqlite** is also included, likely serving as a lightweight alternative for local development or automated testing).
- **Redis (redis):** An in-memory data structure store.
 - Justification: Used for caching, session management, or rate-limiting. Redis operates entirely in memory, offering sub-millisecond read/write speeds that significantly reduce the load on the primary PostgreSQL database compared to storing transient state on disk.

- **SQLAlchemy & Alembic (ORM & Migrations):** Used for managing database queries in Python and tracking schema updates over time.
 - Justification: SQLAlchemy translates Python classes into database tables (Object-Relational Mapping), preventing SQL injection attacks and making database queries database-agnostic. Alembic acts as its migration tool, providing a safe, version-controlled way to alter database schemas over time, which is vastly safer than executing manual SQL scripts.

1.4 Security & Authentication

- **Passlib & Bcrypt (`passlib`, `bcrypt`):** Used for cryptographic hashing of user passwords.
 - Justification: Bcrypt automatically incorporates salting and is designed to be computationally slow, protecting user credentials against brute-force and rainbow table attacks far better than fast hashing algorithms like MD5 or SHA-256.
- **Python-JOSE (`python-jose`):** Used for generating and verifying JSON Web Tokens (JWT).
 - Justification: JWTs allow for stateless authentication. The server does not need to query the database to verify a user's session on every request, which heavily optimizes API response times.

1.5 Testing & Quality Assurance

- **Pytest Suite (`pytest`, `pytest-asyncio`, `pytest-cov`):** The testing framework used for the backend.

- Justification: Pytest requires significantly less boilerplate code than Python's built-in `unittest` module. `pytest-asyncio` allows for native testing of asynchronous FastAPI endpoints, and `pytest-cov` strictly tracks test coverage to ensure code reliability.

1.6 Development & Build Tools

- **Android Studio (Frontend IDE):** The primary integrated development environment.
 - Justification: Provides first-class, official support for Flutter and Dart, alongside powerful built-in device emulators and profiling tools to test UI performance bottlenecks.
- **Uvicorn (`uvicorn`, `uvloop`):** An ASGI web server implementation for Python.
 - Justification: Required to run FastAPI. Paired with `uvloop` (a drop-in replacement for the asyncio event loop written in Cython), it allows the backend to handle thousands of concurrent connections effortlessly.
- **GitHub (Version Control System):** Used for source code management.
 - Justification: Provides a robust, centralized repository for Git. It outpaces alternatives by offering superior integration with CI/CD pipelines (GitHub Actions) and streamlined pull-request workflows for code reviews.

2 Codebase

Repository Structure

The repository is divided into two top-level directories — `mobile/` for the Flutter application and `backend/` for the FastAPI server — alongside shared infrastructure configuration.

Here is the repo github link :

<https://github.com/The-Bread-Omelette/TrackMate/>

```
TrackMate/
├─ mobile/           # Flutter mobile application
├─ backend/          # FastAPI server
├─ docker-compose.yml # Service orchestration
└─ ENDPOINTS.md      # Full API endpoint reference
```

2.1 Mobile — Top-level

The `mobile/` directory follows the standard Flutter project layout. Platform-specific build targets are isolated in their own folders, while all application logic lives under `lib/`.

```
mobile/
├─ lib/              # All Dart source code (core + features)
├─ assets/
│   └─ images/       # App image assets
├─ android/          # Android platform build config
├─ ios/              # iOS platform build config
├─ linux/            # Linux desktop build config
├─ macos/            # macOS desktop build config
├─ windows/          # Windows desktop build config
├─ web/              # Web build config and icons
└─ pubspec.yaml      # Flutter dependencies manifest
```


2.2 Mobile — lib/

Inside `lib/`, the project adopts a **feature-first Clean Architecture** layout. Each feature is a self-contained module with its own data, domain, and presentation layers. Shared utilities and theming are housed in `core/` and `shared/`.

```
lib/
├── main.dart                # App entry point; initialises DI and runs widget
├── core/                   # App-wide infrastructure
│   ├── constants/
│   │   └── api_constants.dart    # Base URL and route constants
│   ├── di/
│   │   └── injection.dart        # GetIt service-locator registration
│   ├── errors/
│   │   ├── failures.dart         # Domain-layer failure types
│   │   └── api_exception.dart    # HTTP error models
│   ├── network/
│   │   └── dio_client.dart        # Dio instance with interceptors & JWT injection
│   ├── router/
│   │   └── app_router.dart        # GoRouter routes, guards, and redirects
│   └── storage/
│       └── token_storage.dart     # Secure JWT read/write (flutter_secure_storage)
├── features/
│   ├── auth/
│   │   ├── data/
│   │   │   ├── datasources/
│   │   │   │   └── auth_remote_datasource.dart
│   │   │   ├── models/
│   │   │   │   └── auth_models.dart
│   │   │   └── repositories/
│   │   │       └── auth_repository_impl.dart
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── user_entity.dart
│   │   │   └── repositories/
│   │   │       └── auth_repository.dart
│   │   └── presentation/
│   │       ├── bloc/
│   │       │   ├── auth_bloc.dart
│   │       │   ├── auth_event.dart
│   │       │   └── auth_state.dart
│   │       └── pages/
```

```
└─ login_page.dart
└─ register_page.dart
└─ onboarding_page.dart
└─ check_email_page.dart

└─ dashboard/
  └─ data/
    └─ dashboard_remote_datasource.dart
  └─ domain/
    └─ entities/
      └─ dashboard_entity.dart
  └─ presentation/
    └─ bloc/
      └─ dashboard_bloc.dart
      └─ dashboard_event.dart
      └─ dashboard_state.dart

└─ fitness/
  └─ services/
    └─ pedometer_service.dart    # Step counter sensor service

└─ workout/
  └─ data/
    └─ workout_remote_datasource.dart
  └─ presentation/
    └─ pages/
      └─ exercise_page.dart
      └─ cardio_timer_page.dart

└─ nutrition/
  └─ data/
    └─ nutrition_remote_datasource.dart
  └─ presentation/
    └─ pages/
      └─ food_logging_page.dart

└─ analytics/
  └─ data/
    └─ analytics_remote_datasource.dart
  └─ presentation/
```

```
└─ pages/
    └─ social_page.dart

└─ messaging/
    └─ data/
        └─ messaging_remote_datasource.dart
    └─ presentation/
        └─ pages/
            ├── conversations_page.dart
            └─ chat_page.dart

└─ trainer/
    └─ data/
        └─ trainer_remote_datasource.dart
    └─ presentation/
        └─ pages/
            ├── find_trainer_page.dart
            ├── coaching_hub_page.dart
            ├── trainer_students_page.dart
            ├── trainer_requests_page.dart
            └─ student_detail_page.dart

└─ calendar/
    └─ presentation/
        └─ pages/
            └─ calendar_page.dart

└─ notifications/
    └─ data/
        └─ notifications_remote_datasource.dart
    └─ presentation/
        └─ pages/
            └─ notifications_page.dart

└─ admin/
    └─ data/
        └─ admin_remote_datasource.dart
    └─ presentation/
        └─ pages/
            └─ admin_dashboard_page.dart
```

```
|
|      └─ admin_users_page.dart
|      └─ admin_trainers_page.dart
|      └─ admin_reports_page.dart
|
└─ shared/                # Reusable UI components and theme
    └─ pages/
        └─ dashboard_page.dart    # Main home dashboard shell
        └─ settings_page.dart     # User settings and preferences
    └─ theme/
        └─ app_theme.dart         # Global MaterialTheme definition
    └─ widgets/
        └─ main_layout.dart       # Scaffold with bottom nav bar
        └─ tm_text_field.dart     # Styled text input component
        └─ role_selector.dart     # User / Trainer role picker widget
```

2.3 Backend — app/

The backend follows a layered structure modelled after standard FastAPI conventions, with separate directories for API routing, database models, Pydantic schemas, and business-logic services.

```
backend/
├── app/
│   ├── main.py                # FastAPI app factory; mounts routers, CORS, middleware
│   ├── api/v1/
│   │   ├── router.py          # APIRouter aggregating all endpoint modules
│   │   ├── deps.py            # Shared dependencies (DB session, current user)
│   │   └── endpoints/
│   │       ├── auth.py        # Register, login, email verification, refresh
│   │       ├── fitness.py      # Steps, calories, workout logs, water intake
│   │       ├── trainer.py      # Trainer profiles, requests, student management
│   │       ├── messaging.py     # Conversations, messages, WebSocket endpoint
│   │       ├── social.py       # Follow/unfollow, activity feed, leaderboard
│   │       ├── notifications.py # Notification list and mark-read
│   │       ├── profile.py      # User profile CRUD
│   │       └── admin.py        # Admin user/trainer management, reports
│   ├── core/
│   │   ├── config.py          # Pydantic Settings (env vars)
│   │   ├── security.py        # JWT creation/decoding, bcrypt hashing
│   │   ├── redis.py           # Redis connection pool
│   │   └── exceptions.py      # Custom HTTP exception classes
│   ├── db/
│   │   └── base.py            # SQLAlchemy async engine and session factory
│   ├── models/                # SQLAlchemy ORM table definitions
│   │   ├── user.py
│   │   ├── profile.py
│   │   ├── fitness.py
│   │   ├── trainer.py
│   │   ├── messaging.py
│   │   ├── social.py
│   │   └── notification.py
│   ├── schemas/               # Pydantic request/response models
│   │   ├── auth.py
│   │   └── profile.py
│   └── ...
```


2.4 Database Schema

TrackMate uses a relational PostgreSQL schema. The key tables and their principal fields are summarised below:

```
-- Core tables

users
├─ id, email, hashed_password, role (user | trainer | admin)
├─ is_verified, is_active
└─ created_at

profiles
├─ user_id (FK → users)
├─ display_name, bio, avatar_url
└─ height, weight, date_of_birth, gender

fitness                                -- Daily activity snapshots
├─ user_id (FK → users), date
├─ steps, calories_burned
└─ water_intake_ml

workout_logs
├─ user_id (FK → users), exercise_name
├─ sets, reps, weight_kg, duration_seconds
└─ logged_at

nutrition_logs
├─ user_id (FK → users), food_name
├─ quantity_g, calories, protein_g, carbs_g, fat_g
└─ logged_at

trainers
├─ user_id (FK → users)
├─ specialisation, experience_years, bio
└─ is_approved

trainer_requests
├─ student_id (FK → users), trainer_id (FK → trainers)
├─ status (pending | accepted | rejected)
└─ created_at

conversations
├─ id, created_at
└─ participants → conversation_participants
```

```

messages
├─ conversation_id (FK → conversations)
├─ sender_id (FK → users), content
└─ sent_at, read_at

social_follows
├─ follower_id (FK → users)
└─ following_id (FK → users)

notifications
├─ user_id (FK → users), type, content
├─ is_read
└─ created_at

```

2.5 BLoC Architecture

TrackMate uses the BLoC (Business Logic Component) pattern throughout the mobile app. Each feature that has significant async logic owns its own Bloc, Events, and States. The flow in every case is: a **Widget** dispatches an **Event** → the **Bloc** performs async work via a repository or datasource → a new **State** is emitted → the **Widget** rebuilds reactively.

AuthBloc

Manages the full authentication lifecycle — from app start session check through registration, OTP verification, onboarding, and logout. It is the source of truth for the router guard in `app_router.dart`, which redirects based on the current `AuthState`.

Event	Trigger	Resulting State(s)
AuthCheckSessionEvent	App launch — Dio tries GET /auth/me	AuthLoadingState → AuthAuthenticatedState (user) or AuthUnauthenticatedState

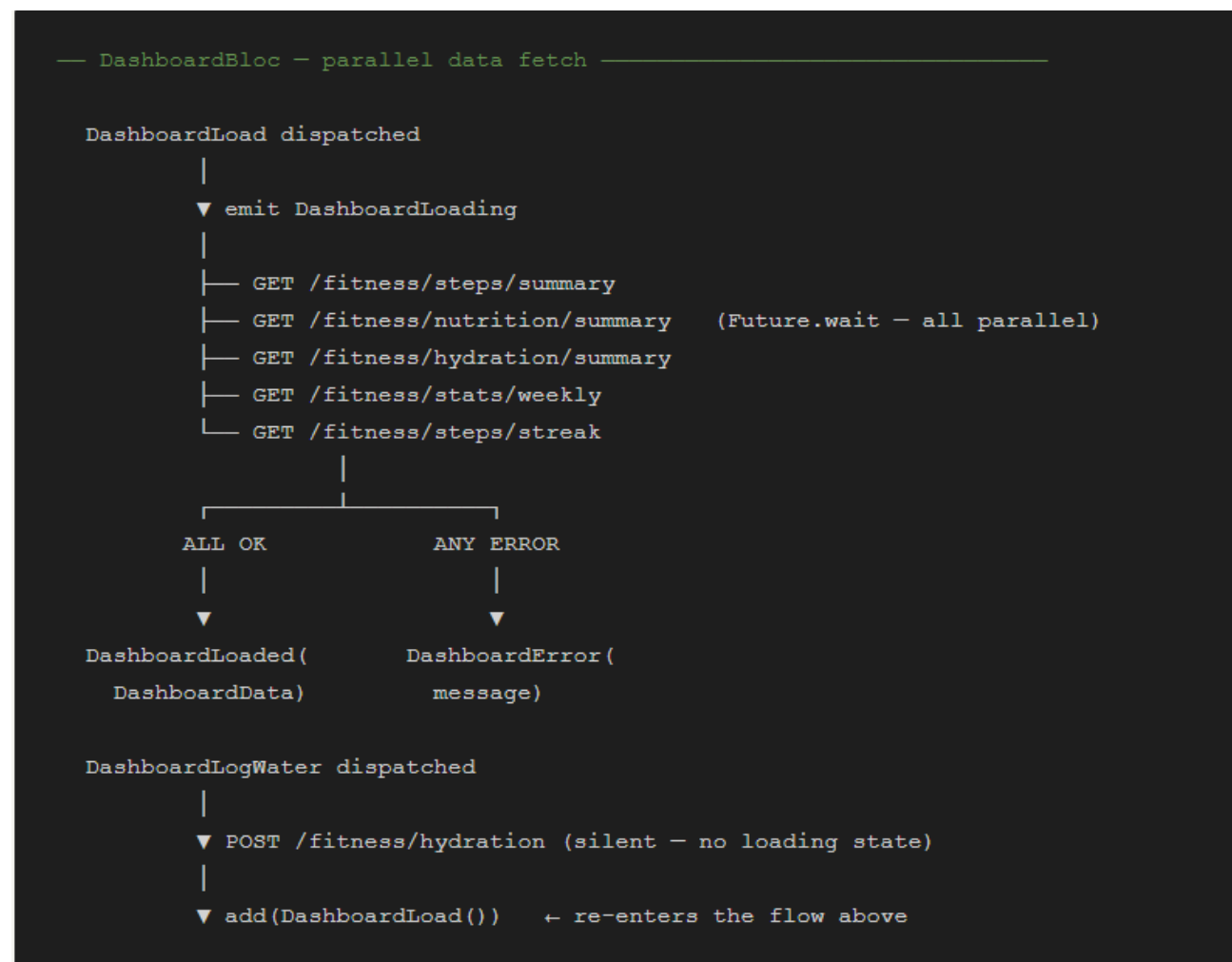
	with stored cookie	
AuthLoginEvent	User taps Login on LoginPage	AuthLoadingState → AuthAuthenticatedState (user); on unverified email: AuthUnverifiedState (email); on other error: AuthErrorState → AuthUnauthenticatedState
AuthRegisterEvent	User submits registration form	AuthLoadingState → AuthRegisteredState (email) (navigates to OTP page); on error: AuthErrorState → AuthUnauthenticatedState
AuthVerifyEmailEvent	User submits 6-digit OTP on CheckEmail Page	AuthLoadingState → AuthOnboardingState (navigates to onboarding); on failure: AuthErrorState → AuthUnverifiedState
AuthResendVerificationEvent	User taps "Resend OTP"	AuthLoadingState → AuthRegisteredState (email)
AuthCompleteOnboardingEvent	User finishes onboarding form and taps Continue	AuthLoadingState → AuthAuthenticatedState (user) (triggers router redirect to dashboard)
AuthLogoutEvent	User taps Logout in Settings	AuthLoadingState → AuthUnauthenticatedState



DashboardBloc

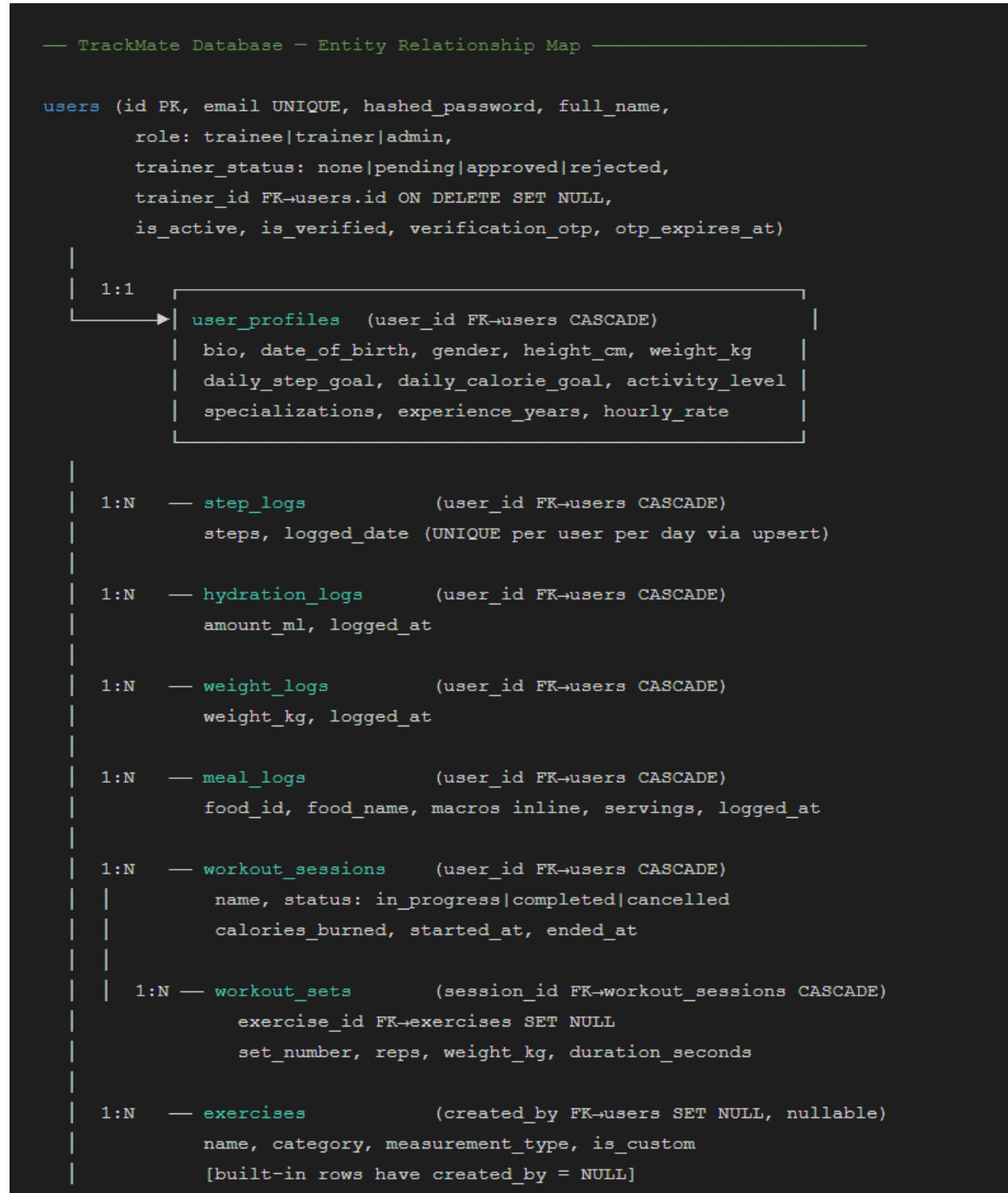
Fetches all dashboard metrics in a single parallel batch on load, and handles inline water logging with an immediate reload to keep the UI fresh.

Event	Trigger	Resulting State(s)
DashboardLoad	Dashboard page enters view; also triggered after DashboardLogWater	DashboardLoading → DashboardLoaded(DashboardData) or DashboardError(message)
DashboardLogWater	User taps a water quantity chip on the dashboard	Fires POST /fitness/hydration silently, then internally dispatches DashboardLoad to refresh all metrics



2.6 Database ER Relationships

All tables use UUID primary keys. Foreign keys are enforced at the database level with explicit ON DELETE behaviours as defined in the Alembic migration. The diagram below shows all 17 tables and their relationships.



```
1:N  — notifications      (user_id FK→users CASCADE)
      type (enum 9 values), title, body, is_read, reference_id

1:N  — posts              (author_id FK→users CASCADE)
      | content, is_deleted
      |
      | 1:N — post_likes   (post_id FK→posts CASCADE,
      |                    user_id FK→users CASCADE)

M:N  — friend_requests
      sender_id FK→users CASCADE
      receiver_id FK→users CASCADE
      status: pending|accepted|rejected

1:N  — trainer_applications (user_id FK→users CASCADE, UNIQUE)
      phone_number, experience_years, about
      specializations, certifications, hourly_rate
      status: pending|approved|rejected

M:N  — trainer_requests
      trainee_id FK→users CASCADE
      trainer_id FK→users CASCADE
      goal, status: pending|accepted|rejected

1:N  — trainer_notes
      trainer_id FK→users CASCADE
      trainee_id FK→users CASCADE
      content

1:N  — trainer_sessions
      trainer_id FK→users CASCADE
      trainee_id FK→users CASCADE
      scheduled_at, duration_minutes, notes

M:N via join table:

— conversations (id, created_at)
```

```
— conversation_members (conversation_id FK CASCADE,
                        user_id FK CASCADE,
                        last_read_at, joined_at)
```

```
├── messages (conversation_id FK CASCADE,
            sender_id FK→users CASCADE,
            reply_to_id FK→messages SET NULL,
            content, status: sent|delivered|read,
            is_deleted, is_pinned)
└── reports
    reporter_id FK→users CASCADE
    reported_user_id FK→users CASCADE
    message_id FK→messages SET NULL (optional)
    post_id FK→posts SET NULL (optional)
    report_type: spam|harassment|inappropriate|other
    status: pending|resolved|dismissed
```

ON DELETE Behaviour Summary

Child Table	FK Column	References	On Delete
users	trainer_id	users.id	SET NULL
user_profiles	user_id	users.id	CASCADE
step_logs	user_id	users.id	CASCADE
hydration_logs	user_id	users.id	CASCADE
weight_logs	user_id	users.id	CASCADE
meal_logs	user_id	users.id	CASCADE

workout_sessions	user_id	users.id	CASCADE
exercises	created_by	users.id	SET NULL
notifications	user_id	users.id	CASCADE
posts	author_id	users.id	CASCADE
post_likes	post_id	posts.id	CASCADE
conversation_members	conversation_id	conversations.id	CASCADE
messages	reply_to_id	messages.id	SET NULL
trainer_requests	trainee_id / trainer_id	users.id	CASCADE
trainer_applications	user_id	users.id	CASCADE
reports	message_id / post_id	messages / posts	SET NULL

2.7 API Endpoint Specifications

All endpoints are served under the base URL <https://trackmate-tgmy.onrender.com> and prefixed with `/api/v1`. Authentication uses `httpOnly` cookie-based JWT tokens set automatically on login and sent by Dio on every subsequent request. A `POST /api/v1/auth/refresh` is called automatically by the Dio interceptor on any 401 response.

Auth legend — **Yes**: requires valid `access_token` cookie · **No**: public endpoint · **Trainer only** / **Admin only**: role-gated, returns 403 if role does not match.

Authentication

POST /api/v1/auth/register

Register a new account. Account is inactive until email is verified. Sends a 6-digit OTP that expires in 10 minutes.

Field	Type	Notes
email	string	Required
password	string	Min 8 chars, ≥1 uppercase, ≥1 digit
full_name	string	Min 2 chars
apply_as_trainer	boolean	Optional, default false

Auth: No | **Success:** 201 {"message": "Account created..."} | **Errors:** 409 email already registered

POST /api/v1/auth/verify-email

Verify email with the OTP. Activates the account, logs the user in, and sets `access_token` + `refresh_token` httpOnly cookies.

Field	Type	Notes
email	string	Required
otp	string	6-digit code from email

Auth: No | **Success:** 200 user object + tokens | **Errors:** 401 invalid/expired OTP, 409 already verified

POST /api/v1/auth/resend-verification

Resend a fresh OTP. Invalidates the previous code.

Auth: No | **Body:** {"email": "..."} | **Success:** 200 | **Errors:** 404 not found, 409 already verified

POST /api/v1/auth/login

Login with email and password. Role is derived server-side.

Auth: No | **Success:** 200 user + tokens | **Errors:** 401 wrong credentials, 403 deactivated or unverified (auto-resends OTP)

POST /api/v1/auth/logout

Clears session cookies. **Auth:** Yes | **Success:** 200

POST /api/v1/auth/refresh

Refresh access token using the `refresh_token` cookie. Called automatically by the Dio interceptor on 401. **Auth:** Cookie (`refresh_token`) | **Success:** 200 new token pair | **Errors:** 401 token missing/expired

GET /api/v1/auth/me

Returns the currently authenticated user object. **Auth:** Yes | **Success:** 200 user object

Profile

GET /api/v1/profile/me

Get current user's full profile including biometrics, goals, and calculated TDEE (Mifflin-St Jeor formula). Profile is auto-created on first fetch. **Auth:** Yes | **Success:** 200

PUT /api/v1/profile/me

Update profile fields. All fields are optional — send only what needs changing.

Field	Type	Validation
bio	string	—
date_of_birth	ISO datetime	—

gender	enum	male female other prefer_not_to_say
height_cm	float	50–300
weight_kg	float	20–500
daily_step_goal	int	1000–100000
daily_calorie_goal	int	500–10000
activity_level	enum	sedentary lightly_active moderately_active very_active extra_active

Auth: Yes | **Success:** 200 updated profile with recalculated TDEE

GET /api/v1/profile/users/search?q={query}

Search users by name or email. Returns max 20 results, excludes self, only active + verified users.

Auth: Yes | **Success:** 200 array of user objects

Trainer

POST /api/v1/trainer/apply

Submit a trainer application (trainees only). Creates a pending application. **Auth:** Yes | **Success:** 201 {"application_id", "status": "pending"} | **Errors:** 409 already applied

GET /api/v1/trainer/available

Browse all admin-approved trainers with profile info. **Auth:** Yes | **Success:** 200 array of trainer profiles

POST /api/v1/trainer/request

Send a coaching request to a trainer. Body: {"trainer_id", "goal"}. Side effect: notification sent to trainer. **Auth:** Yes | **Success:** 201 | **Errors:** 409 already has trainer or pending request

GET /api/v1/trainer/my-trainer

Get the current user's assigned trainer (or `null`). **Auth:** Yes

GET /api/v1/trainer/my-sessions

Get sessions scheduled for the current trainee by their trainer. **Auth:** Yes

GET /api/v1/trainer/students

List all assigned students with adherence stats: `adherence`, `workout_score`, `streak`, `needs_attention`. **Auth:** Trainer only

GET /api/v1/trainer/students/{trainee_id}

Detailed student view including coaching notes. **Auth:** Trainer only | **Errors:** 404 student not assigned to this trainer

POST /api/v1/trainer/students/{trainee_id}/notes

Add a coaching note. Body: `{"content": "..."}.` **Auth:** Trainer only | **Success:** 201

GET /api/v1/trainer/students/{trainee_id}/workouts

View student's workout history. Query: `limit` (default 20, max 100). **Auth:** Trainer only

GET /api/v1/trainer/students/{trainee_id}/nutrition

View student's nutrition for a date. Query: `date` (ISO, defaults to today). Returns summary + meals array. **Auth:** Trainer only

GET /api/v1/trainer/students/{trainee_id}/stats

View student's weekly stats, streak, step history, and weight trend. **Auth:** Trainer only

GET /api/v1/trainer/stats

Trainer dashboard: `total_students`, `avg_adherence`, `needs_attention` count. **Auth:** Trainer only

GET /api/v1/trainer/requests

List pending trainee requests. **Auth:** Trainer only

PUT /api/v1/trainer/requests/{request_id}

Accept or reject a request. Body: `{"accept": true}`. Side effect: on accept, sets `trainer_id` on trainee and sends notification. **Auth:** Trainer only

GET /api/v1/trainer/calendar

Get upcoming scheduled sessions for the trainer. **Auth:** Trainer only

POST `/api/v1/trainer/calendar/sessions`

Schedule a session with a student. Body: `{"trainee_id", "scheduled_at", "duration_minutes", "notes"}`. **Auth:** Trainer only | **Errors:** 403 trainee not assigned to this trainer

Admin

GET `/api/v1/admin/stats`

Platform-wide stats: total users, active trainers, active WebSocket sessions, pending reports, pending trainer applications, growth rate. **Auth:** Admin only

GET `/api/v1/admin/users`

List all users. Query params: `role filter`, `page`, `limit` (max 100). **Auth:** Admin only

GET `/api/v1/admin/users/{user_id}`

Full user detail including profile. **Auth:** Admin only

PUT `/api/v1/admin/users/{user_id}/deactivate / activate`

Toggle user account status. Deactivated users cannot log in. **Auth:** Admin only | **Success:** 200 confirmation message

GET `/api/v1/admin/trainer-applications`

List applications with optional `status filter` (`pending` | `approved` | `rejected`). Returns summary counts + application array. **Auth:** Admin only

POST `/api/v1/admin/trainer-applications/{user_id}/approve`

Approve or reject application. Body: `{"user_id", "approve": true}`. Side effect: sends approval/rejection email. **Auth:** Admin only

GET `/api/v1/admin/reports`

List reports with optional `status filter`. Returns summary + reports array including reporter, reported user, and linked message/post. **Auth:** Admin only

PUT `/api/v1/admin/reports/{report_id}/resolve`

Resolve or dismiss a report. Body: `{"dismiss": false}`. **Auth:** Admin only | **Success:** 200 updated status

GET `/api/v1/admin/reports/flagged-messages`

Pending reports specifically about messages — includes full message content. **Auth:** Admin only

GET /api/v1/admin/summary/{user_id}

Generate a weekly fitness summary for a user. Returns narrative text + structured metrics (adherence %, workouts, calorie balance, streak). **Auth:** Admin only

Messaging — REST

POST /api/v1/messaging/conversations

Start a conversation with another user, or return the existing one. Body: {"other_user_id"}. **Auth:** Yes | **Success:** 201 conversation_id + other_user | **Errors:** 404 user not found/inactive

GET /api/v1/messaging/conversations

List all conversations sorted by most recent message. Includes last message preview and unread count. **Auth:** Yes

GET /api/v1/messaging/conversations/{conversation_id}/messages

Paginated message history. Query: before (ISO datetime cursor), limit (default 50, max 100). Returns oldest-first within the page. **Auth:** Yes | **Errors:** 403 not a member, 404 not found

PUT /api/v1/messaging/conversations/{conversation_id}/read

Mark all messages as read. Side effect: if the other user is online, sends a messages_read WebSocket event. **Auth:** Yes

DELETE /api/v1/messaging/messages/{message_id}

Soft-delete a message (sender only). **Auth:** Yes | **Errors:** 403 not sender, 404 not found

GET /api/v1/messaging/users/{user_id}/presence

Get online status and last seen timestamp. **Auth:** Yes | **Response:** {"user_id", "online": true, "last_seen"}

POST /api/v1/messaging/ws-ticket

Get a one-time WebSocket auth ticket. Valid for 30 seconds, single use. Must be called immediately before opening the WebSocket. **Auth:** Yes | **Response:** {"ticket": "..."}

Messaging — WebSocket

WS /api/v1/messaging/ws?ticket={ticket}

Authenticated via one-time ticket from `POST /messaging/ws-ticket`. On connect, the server pushes an `unread_summary` if there are pending messages. Send a `heartbeat` every 30 seconds to maintain presence.

Direction	Type	Key Fields	Description
Client → Server	<code>send_message</code>	<code>conversation_id</code> , <code>content</code>	Send a new message
Client → Server	<code>mark_read</code>	<code>conversation_id</code>	Mark all messages read
Client → Server	<code>typing</code>	<code>conversation_id</code>	Show typing indicator; auto-expires 5s
Client → Server	<code>heartbeat</code>	—	Keep-alive; maintains Redis presence
Server → Client	<code>new_message</code>	<code>message object</code>	Incoming message; status: sent → delivered → read
Server → Client	<code>messages_read</code>	<code>conversation_id</code> , <code>read_by</code>	Other user read your messages
Server → Client	<code>typing</code>	<code>conversation_id</code> , <code>user_id</code>	Other user is typing
Server → Client	<code>unread_summary</code>	<code>conversations[]</code> with unread counts	Pushed on connect if unread messages exist

Server → Client	user_online / user_offline	user_id	Presence change for a contact
Server → Client	heartbeat_ack	—	Confirms heartbeat received
Server → Client	error	message	Protocol error (e.g. not a conversation member)

Close codes: 4001 — invalid/expired ticket or inactive user.

Social

POST /api/v1/social/friends/request/{user_id}

Send a friend request. Side effect: notification sent to receiver. **Auth:** Yes | **Success:** 201 | **Errors:** 409 already friends or pending

PUT /api/v1/social/friends/request/{request_id}

Accept or reject a friend request (receiver only). Body: {"request_id", "accept": true}. Side effect on accept: notification to sender. **Auth:** Yes

GET /api/v1/social/friends/requests

List incoming pending requests. **Auth:** Yes

GET /api/v1/social/friends

List all accepted friends. **Auth:** Yes

DELETE /api/v1/social/friends/{friend_id}

Remove a friend. **Auth:** Yes | **Success:** 200

GET /api/v1/social/feed

Posts from friends and self, most recent first. Each post includes like_count and liked_by_me. **Auth:** Yes

POST /api/v1/social/posts

Create a post. Body: {"content": "..."}.
Auth: Yes | **Success:** 201

DELETE /api/v1/social/posts/{post_id}

Soft-delete (author only). **Auth:** Yes | **Errors:** 403 not author

POST /api/v1/social/posts/{post_id}/like

Toggle like. Returns {"liked": true/false}. Side effect: notification to post author on like. **Auth:** Yes

POST /api/v1/social/report/message

Report a message to admin. Body: {"message_id", "reported_user_id", "report_type", "body"}.
Types: spam | harassment | inappropriate | other. **Auth:** Yes | **Success:** 201

GET /api/v1/social/leaderboard

Friends ranked by total steps in the last 7 days. Includes is_me flag on the current user's entry.
Auth: Yes

Notifications

GET /api/v1/notifications

List notifications, most recent first (max 50). Query: unread_only=true. Returns unread_count + notifications array.

Notification type values: friend_request | friend_accepted | trainer_request | trainer_accepted | trainer_rejected | trainer_approved | new_message | post_like | system. **Auth:** Yes

PUT /api/v1/notifications/{notification_id}/read

Mark a single notification as read. **Auth:** Yes

PUT /api/v1/notifications/read-all

Mark all notifications as read. **Auth:** Yes

DELETE /api/v1/notifications/{notification_id}

Delete a notification. **Auth:** Yes

Fitness

GET /api/v1/fitness/exercises?q={term}

Search exercise library (built-in + user's custom). Returns id, name, category, measurement_type, is_custom. **Category:** cardio | strength | flexibility | balance | other. **Measurement:** reps | time. **Auth:** Yes

POST /api/v1/fitness/exercises

Create a custom exercise. Body: {"name", "category", "measurement_type", "description"}. **Auth:** Yes | **Success:** 201

POST /api/v1/fitness/workouts/sessions

Start a new workout session. Body: {"name": "Push Day"}. **Auth:** Yes | **Success:** 201 {"session_id", "started_at"}

PUT /api/v1/fitness/workouts/sessions/{session_id}/finish

Finish a session (status → completed). Body: {"notes", "calories_burned"}. **Auth:** Yes | **Success:** 200 full session with all sets

GET /api/v1/fitness/workouts/sessions

Completed workout history. Query: limit (default 20, max 100). **Auth:** Yes

GET /api/v1/fitness/workouts/sessions/{session_id}

Specific session with all sets, each including exercise name, reps, weight, and duration. **Auth:** Yes

POST /api/v1/fitness/workouts/sessions/{session_id}/sets

Log a set. Use reps + weight_kg for strength or duration_seconds for time-based. **Auth:** Yes | **Success:** 201

DELETE /api/v1/fitness/workouts/sets/{set_id}

Delete a set. **Auth:** Yes | **Success:** 200

GET /api/v1/fitness/foods/search?q={term}

Search Open Food Facts. Results biased toward Indian products. Query: q (min 2 chars), page. Returns macros per 100g + serving info. **Auth:** Yes

GET /api/v1/fitness/foods/barcode/{barcode}

Look up a food by barcode. **Auth:** Yes | **Errors:** 404 not in Open Food Facts

POST /api/v1/fitness/nutrition/meals

Log a meal. Macros are passed inline from the food search result and stored directly (no FK to a products table). Body includes food name, macros per 100g, serving size, and servings count.

logged_at defaults to now. **Auth:** Yes | **Success:** 201

GET /api/v1/fitness/nutrition/meals?date={date}

All meals logged on a date (defaults to today). Each entry includes computed totals for the serving.

Auth: Yes

GET /api/v1/fitness/nutrition/summary?date={date}

Daily nutrition totals: total_calories, total_protein_g, total_carbs_g, total_fat_g, meal_count.

Auth: Yes

DELETE /api/v1/fitness/nutrition/meals/{meal_id}

Delete a meal entry. **Auth:** Yes

POST /api/v1/fitness/steps

Log steps for a date (upserts — calling again for the same date updates the record). Body: {"steps", "logged_date"}. **Auth:** Yes | **Success:** 200

GET /api/v1/fitness/steps/summary

Today's steps vs goal. Returns steps, goal, percentage, remaining. **Auth:** Yes

GET /api/v1/fitness/steps/history?days={n}

Step history for last N days (default 7, max 90). **Auth:** Yes

GET /api/v1/fitness/steps/streak

Consecutive days the step goal was met. Returns {"streak_days": n}. **Auth:** Yes

POST /api/v1/fitness/hydration

Log water intake. Body: {"amount_ml": 500}. **Auth:** Yes | **Success:** 201

GET /api/v1/fitness/hydration/summary

Today's total water: total_ml, total_l, goal_ml, percentage. **Auth:** Yes

GET /api/v1/fitness/stats/weekly

Weekly summary: workouts_completed, calories_burned, total_steps, step_goal_days_met, streak_days. **Auth:** Yes

3 Completeness

1. Implemented Features for Trainee

- 1.1. User Authentication & Role Setup :** F1 and U1 have been implemented according to the SRS document. The system successfully allows users to sign up, verify their identity via OTP, input biometric data (gender, DOB, height, weight), and set initial fitness goals. The login mechanism using email and password is in place, alongside settings to manage profiles and apply for trainer roles.

The image displays two side-by-side mobile application screens for the Trackmate app.

Left Screen: Create Account

- Header: TRACKMATE logo, "Create Account", and the tagline "Start your fitness journey today".
- Form fields:
 - Full Name: AYUSH MANGAL
 - Email: ayushmangal2007@gmail.com
 - Password: [masked]
 - Confirm Password: [masked]
- Buttons: "Create Account" (blue) and "Already have an account? Sign in" (link).

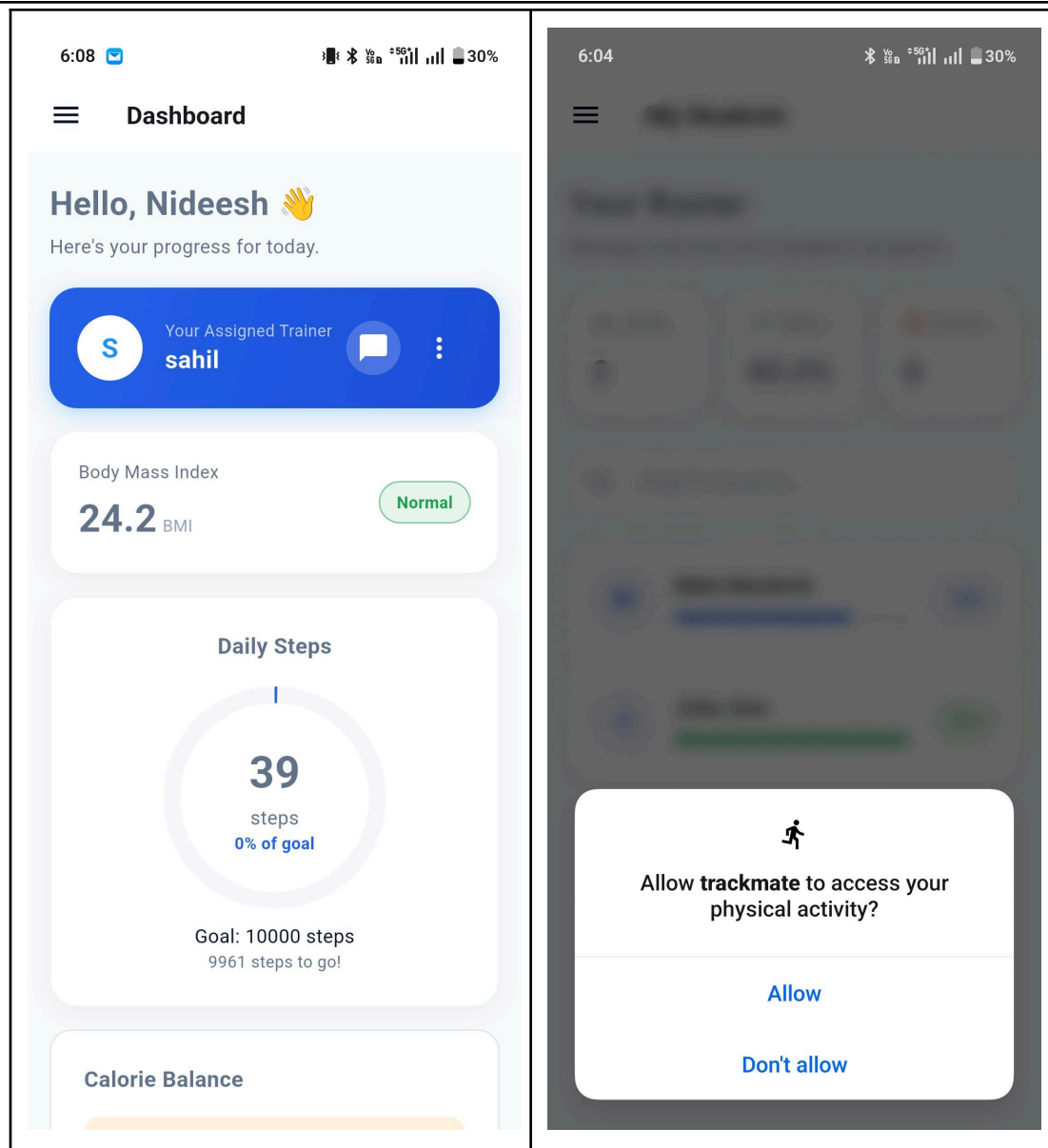
Right Screen: Login

- Header: TRACKMATE logo, "Login", and the tagline "Sign in to continue your fitness journey".
- Form fields:
 - Email: krishnaa24@gmail.com
 - Password: Deepakkanu@1
- Buttons: "Login" (blue) and "Forgot password?" (link).
- Footer: "Don't have an account? Sign up" (link).

Check your email We sent a 6-digit code to aditikararwal29@gmail.com Verify Email Didn't receive the code? Resend Back to login	TRACKMATE Let's get started Tell us about yourself. Gender Male ▼ Date of Birth 1/1/1995 Next >
---	--

TRACKMATE	
Your measurements Used to accurately calculate your calorie needs. Height (cm) 170 Weight (kg) 70	Set your goals We'll track your daily progress towards these metrics. Activity Level Moderately Active (3-5 days/week) ▼ Daily Step Goal 10000 Daily Calorie Goal 2000
< Back Next >	< Back Complete

- 1.2. Step Tracking:** F2, U8 and U4 have been implemented as per the SRS. It is able to accurately track the step count of the user using hardware sensors. It also works even when the app is closed. The step count is updated in the dashboard in near real time.



- 1.3. Meal management:** F3 and U2 have been implemented according to the SRS document. Users can search for food, log meals, and view real-time updates to their macronutrient targets (Protein, Carbs, Fats).

← Food Logging

Today's Macros

993 kcal

Protein

55.1g / 150g

Carbs

149.2g / 200g

Fats

12.5g / 65g

Water Intake

1550 / 2500 ml

[Change water goal](#)

Add: 250 ml

Log Amount

← Food Logging

sa

Search Results

 Dosa

168 kcal · 100g

+

 Sambar

70 kcal · 100g

+

 Rasam

35 kcal · 100g

+

 Samosa

262 kcal · 100g

+

 Paneer Butter Masala

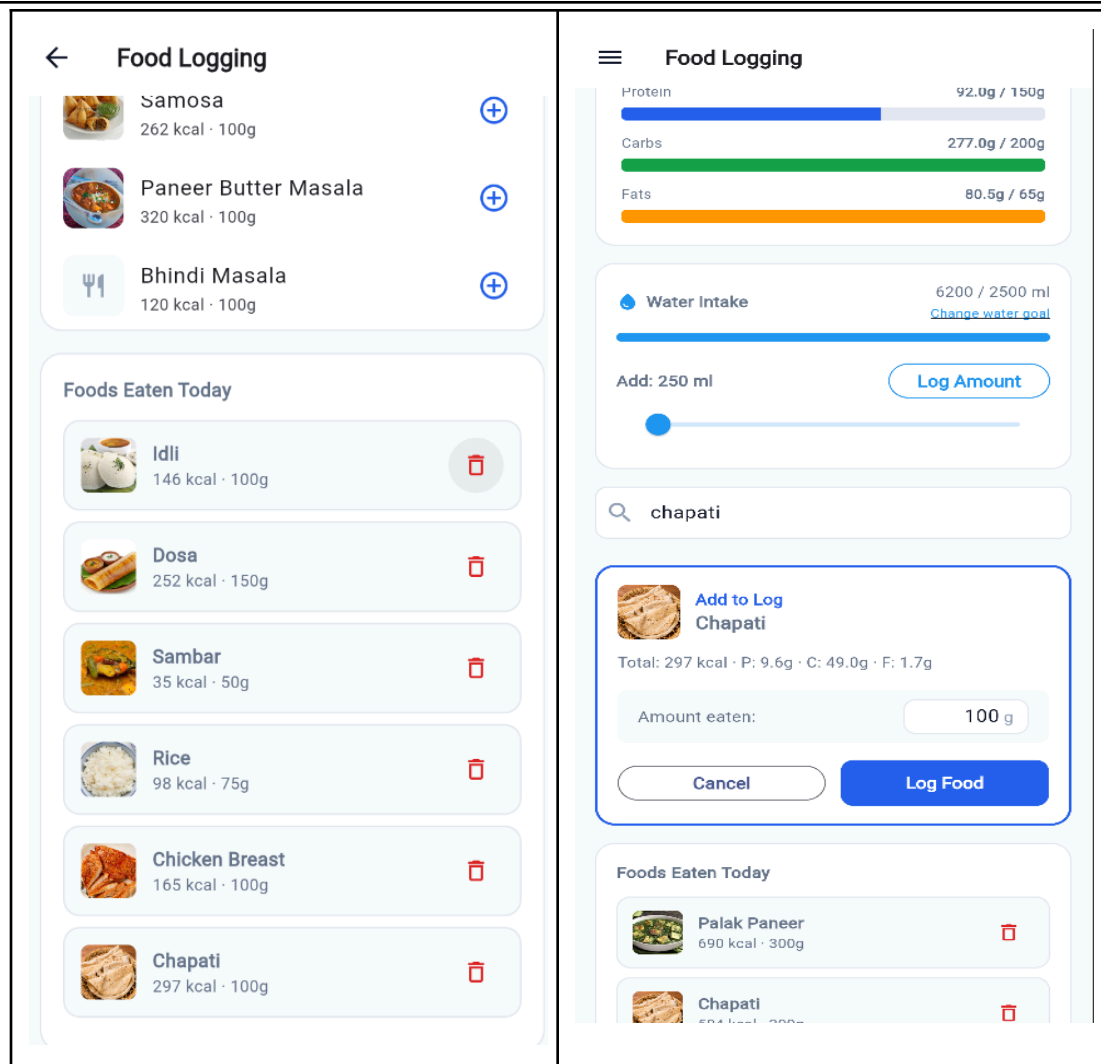
320 kcal · 100g

+

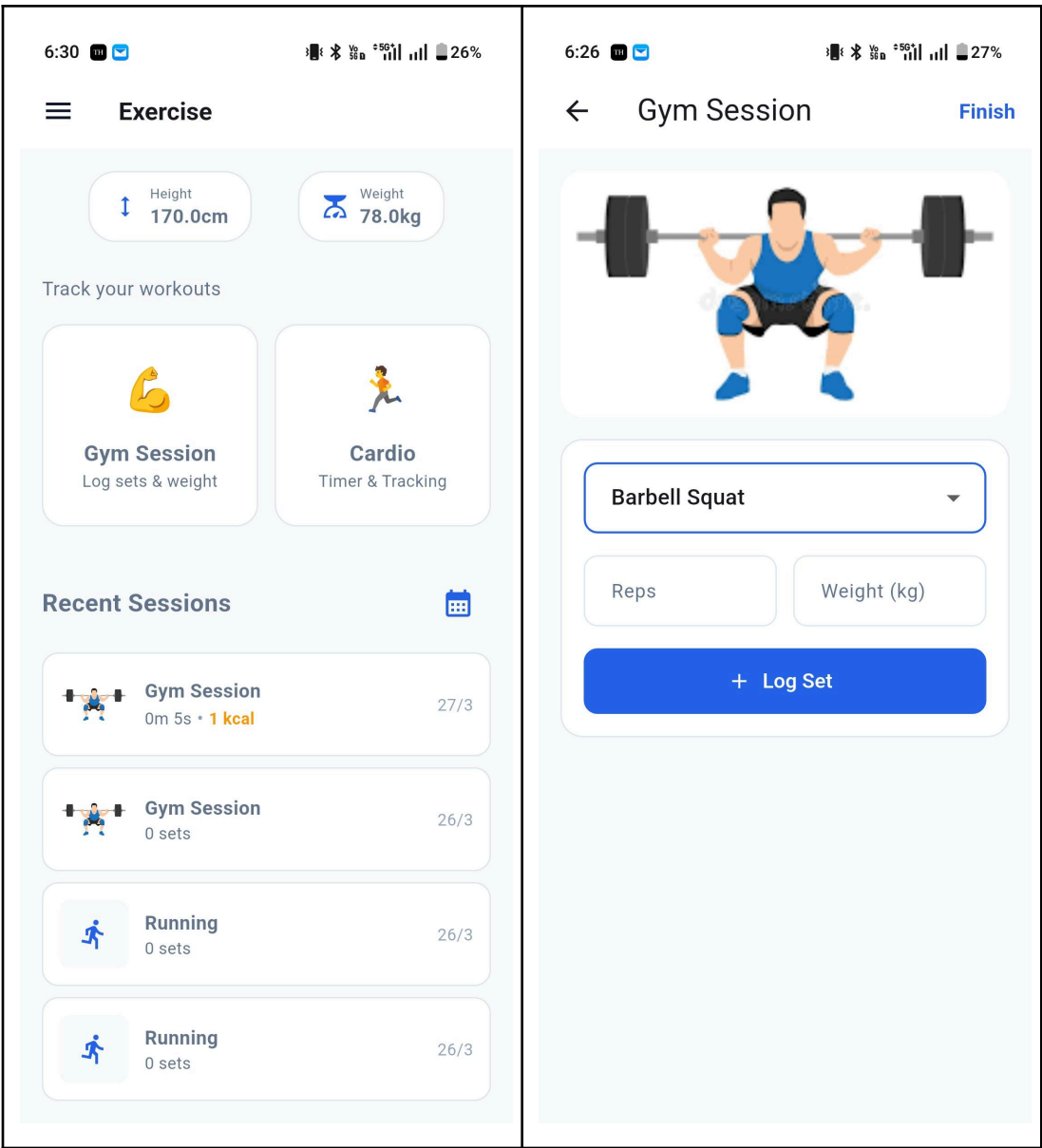
 Bhindi Masala

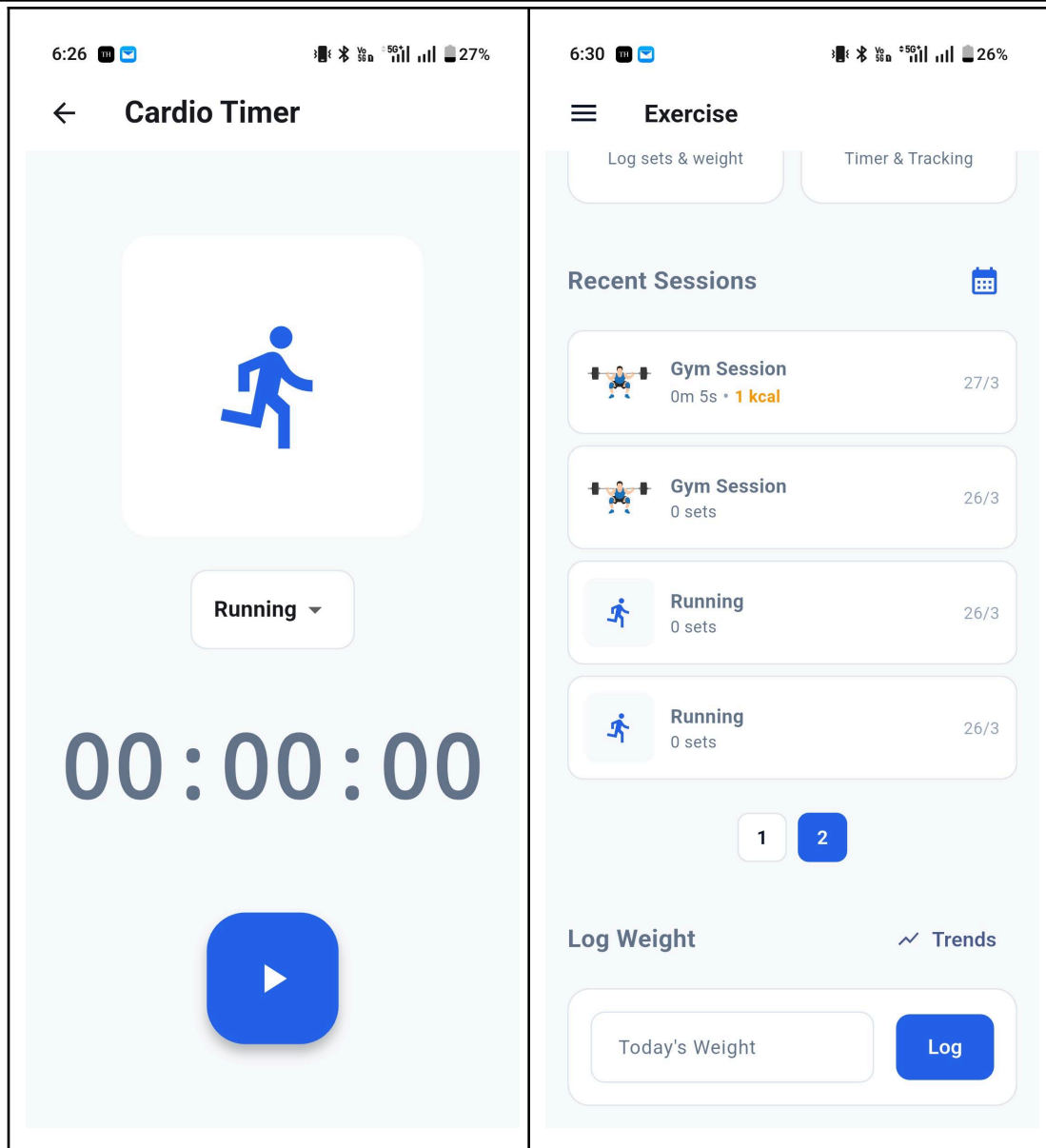
120 kcal · 100g

+

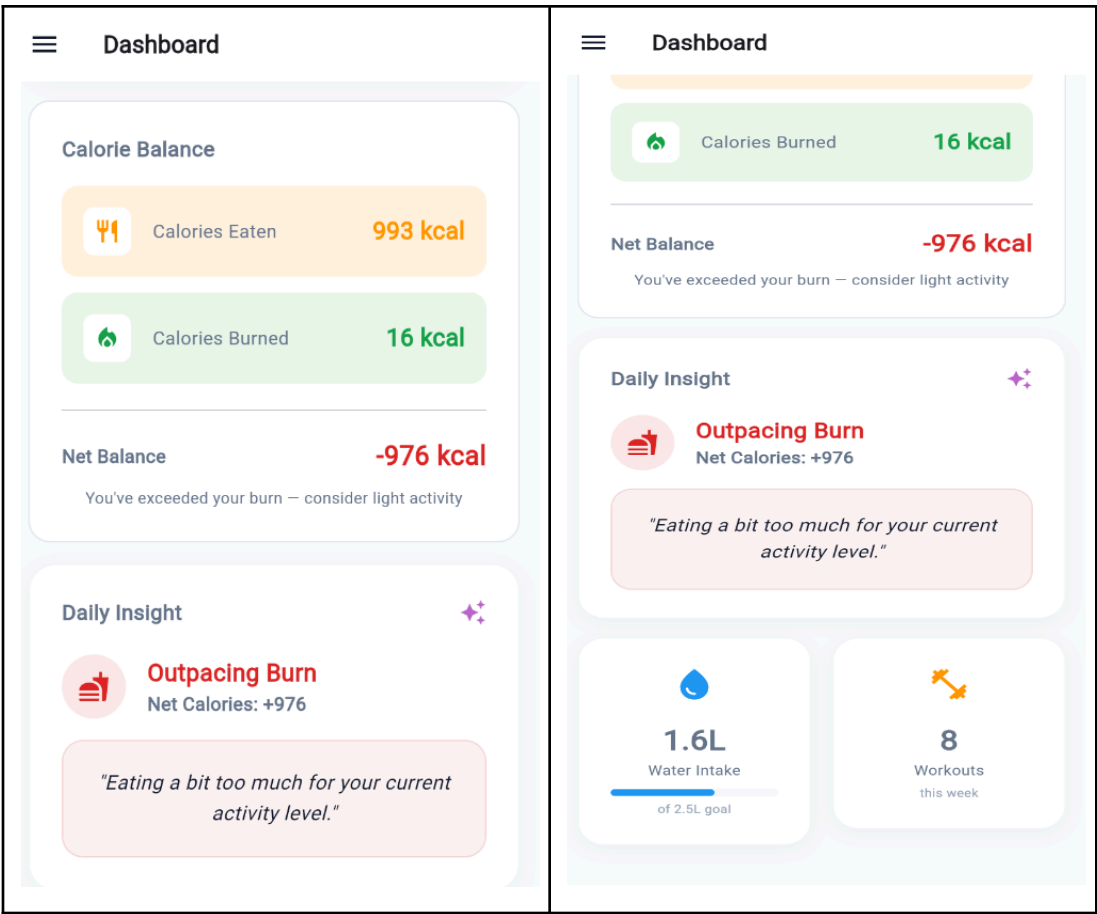


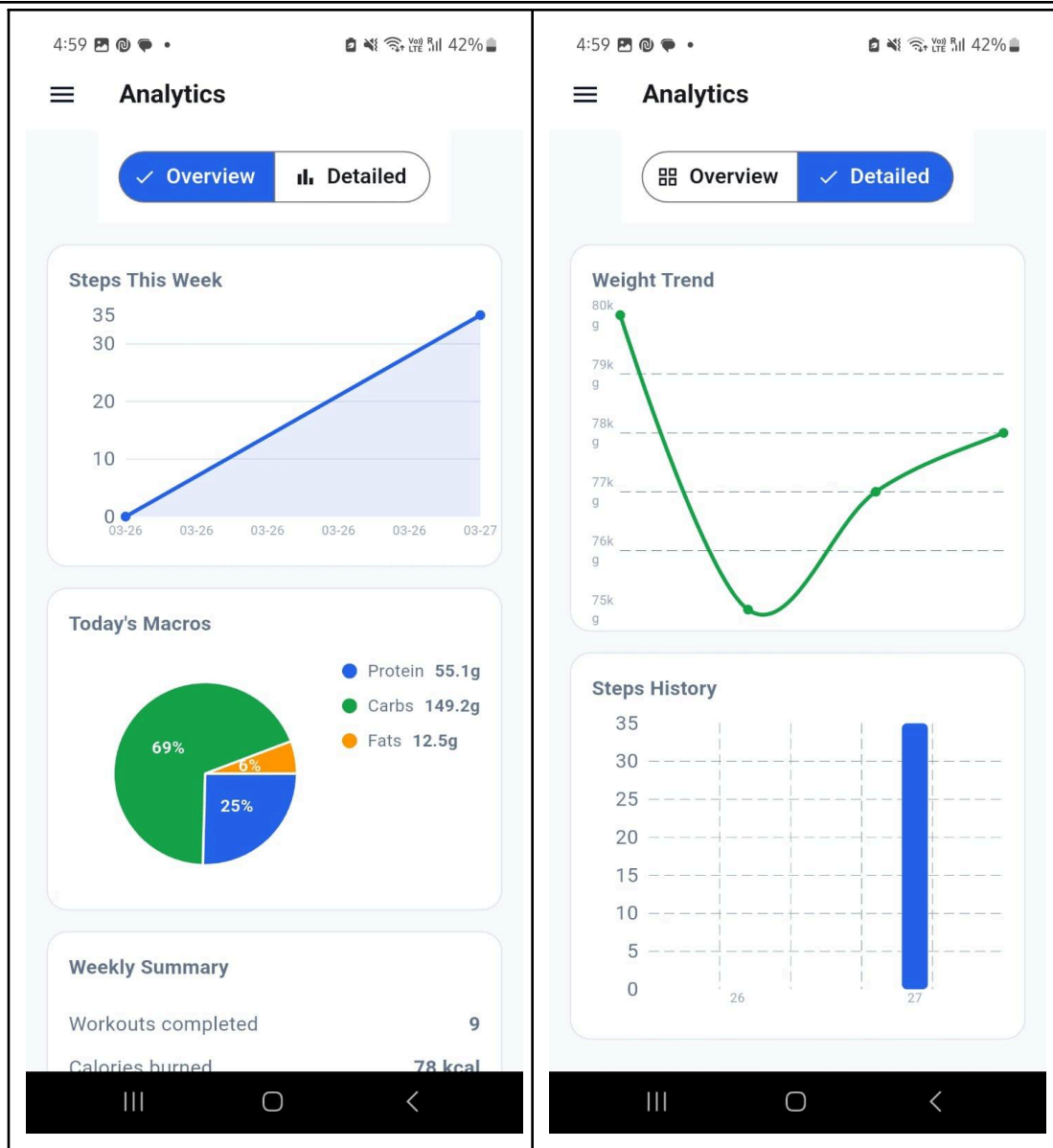
- 1.4. Workout Execution:** F4, F5 and U3 have been implemented according to the SRS document. The app supports structured workout logging. Users can track gym sessions (logging exercise types, sets, reps, and weights) and cardio sessions utilizing a built-in stopwatch. Historical trends, including recent sessions, step history, and weight trends, are visualized for the user.



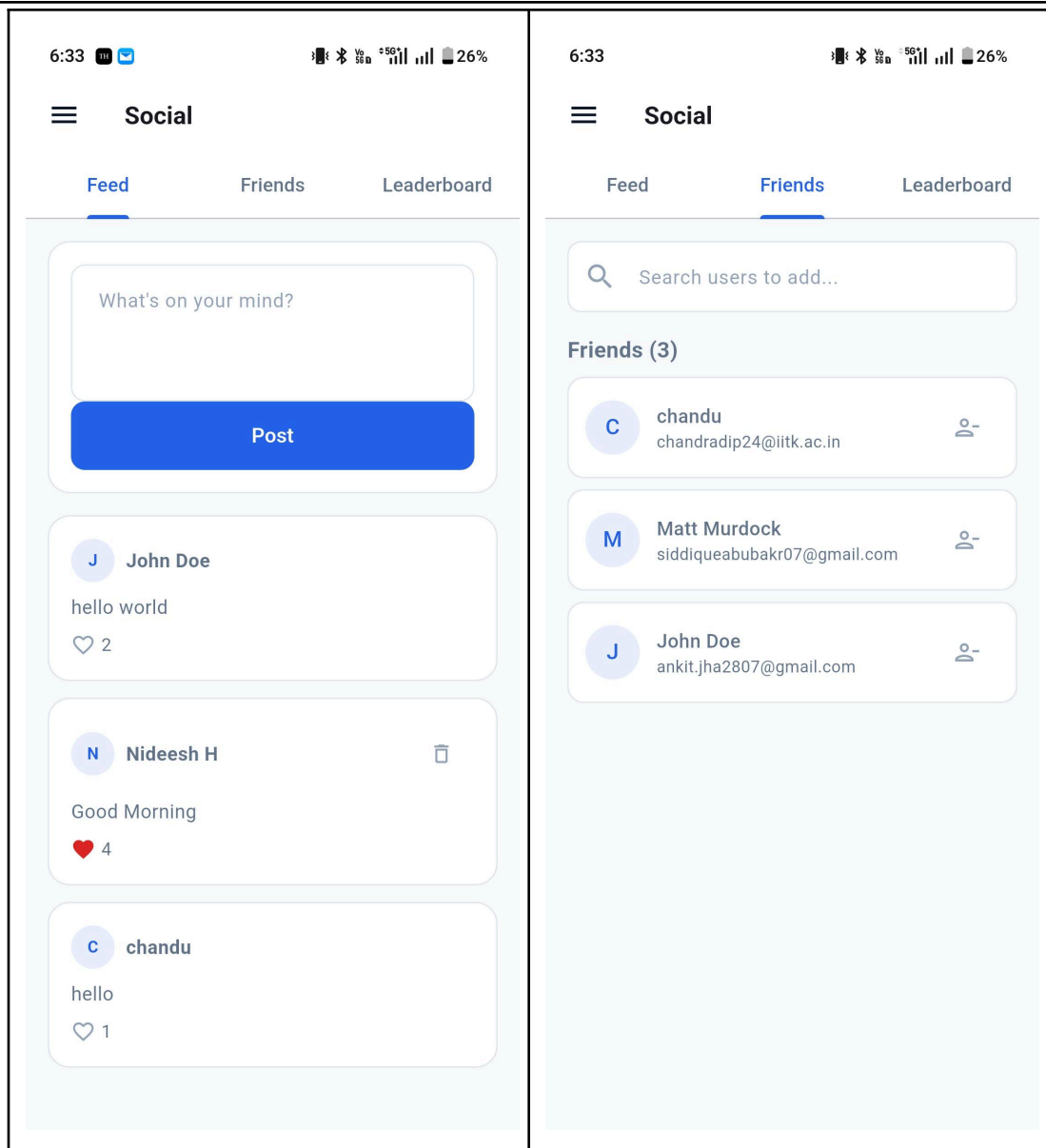


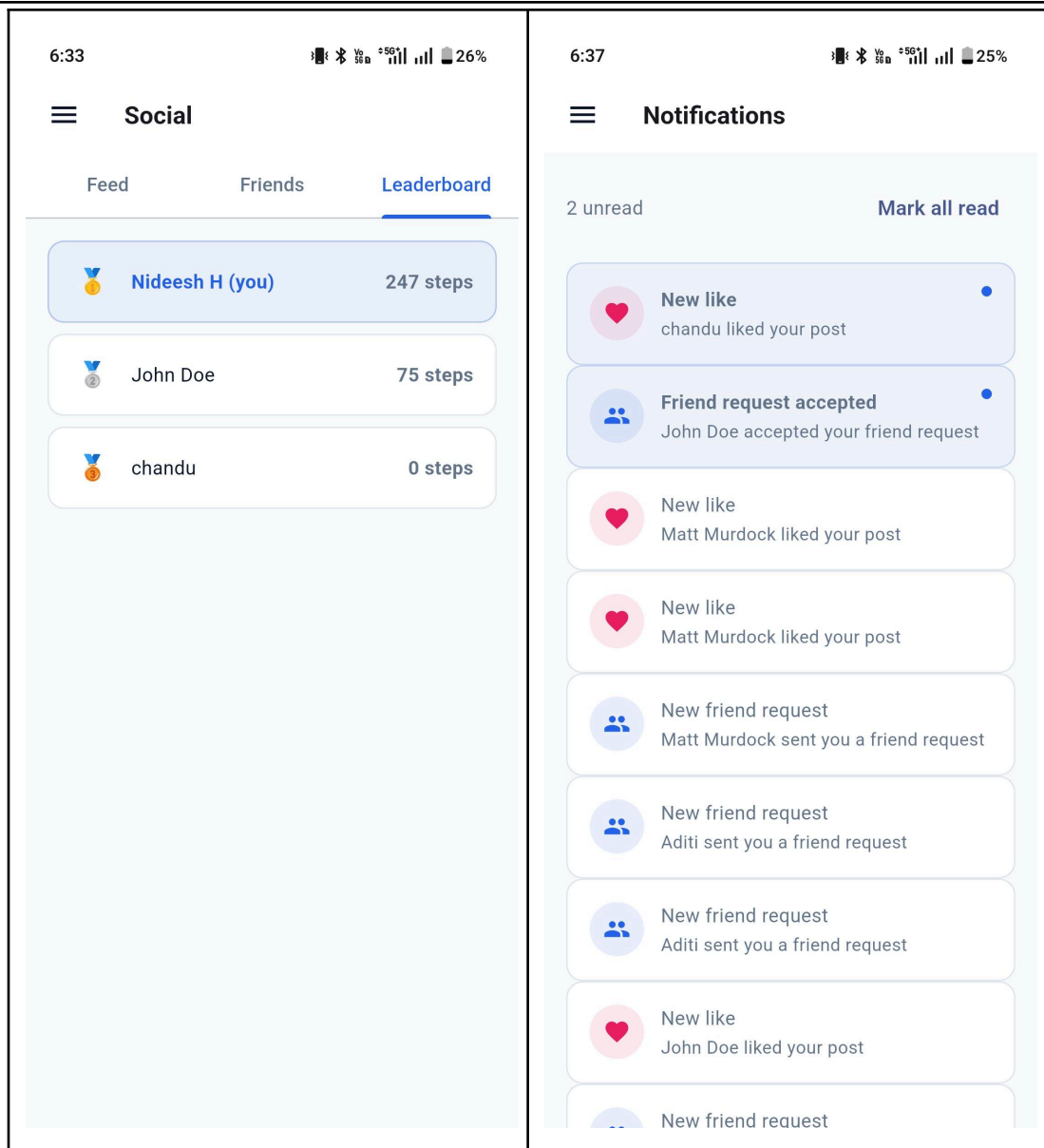
- 1.5. Progress visualization:** F5, U7, U2 and U3 have been implemented according to the SRS document . The interface compares calories consumed and burned, highlighting the net balance with intuitive color cues, green for a deficit (more burned than consumed) and red for a surplus. A “Daily Insights” section presents essential nutritional information, along with water intake tracking and weekly workout logs. The Analytics tab provides more detailed insights, including a weekly weight trend, a steps graph, a pie chart of today’s macronutrients, and a bar chart showing step history. Lastly, the Weekly Summary showcases total workouts, calories burned, steps taken, goal completion status, and current streaks.



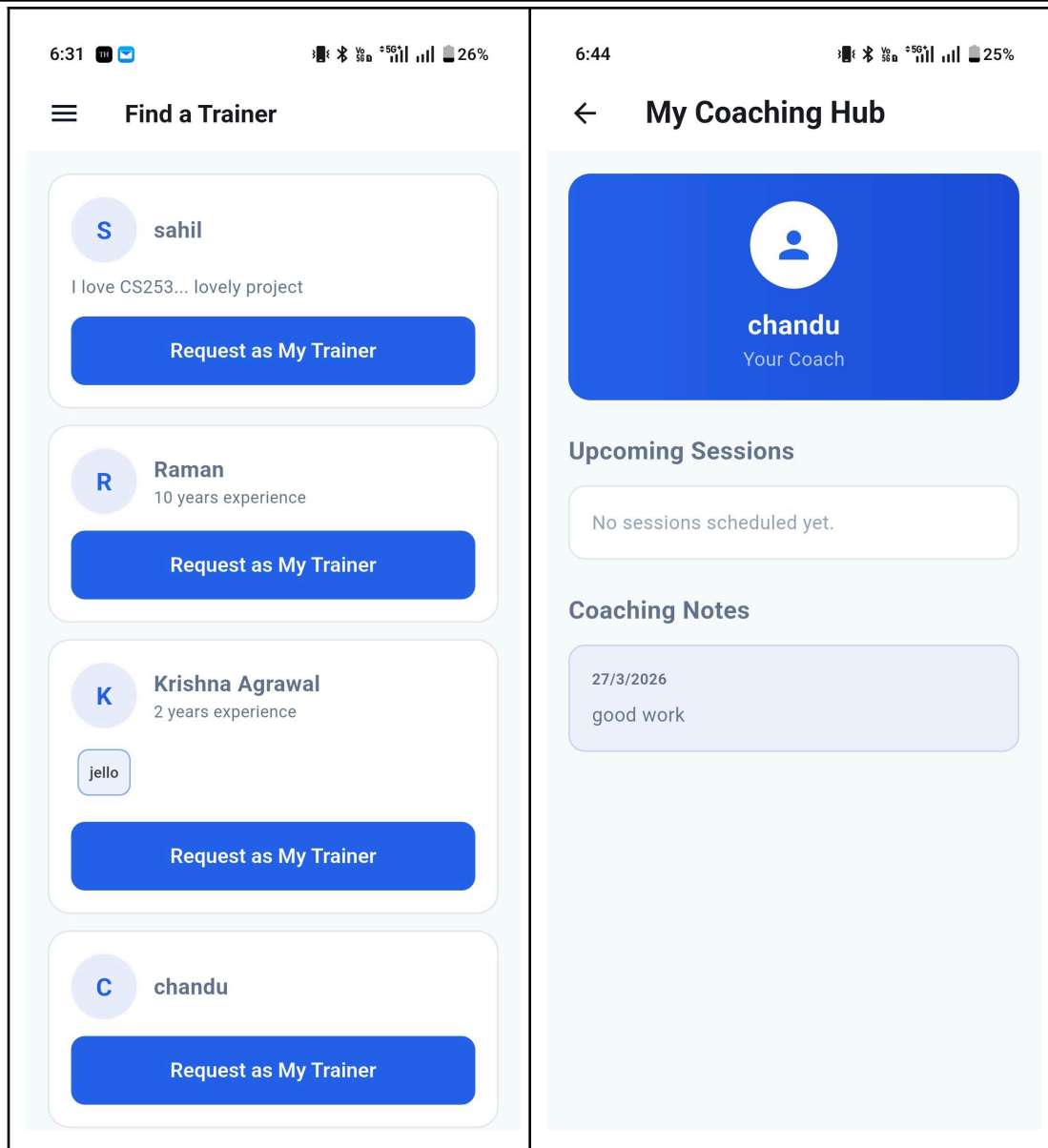


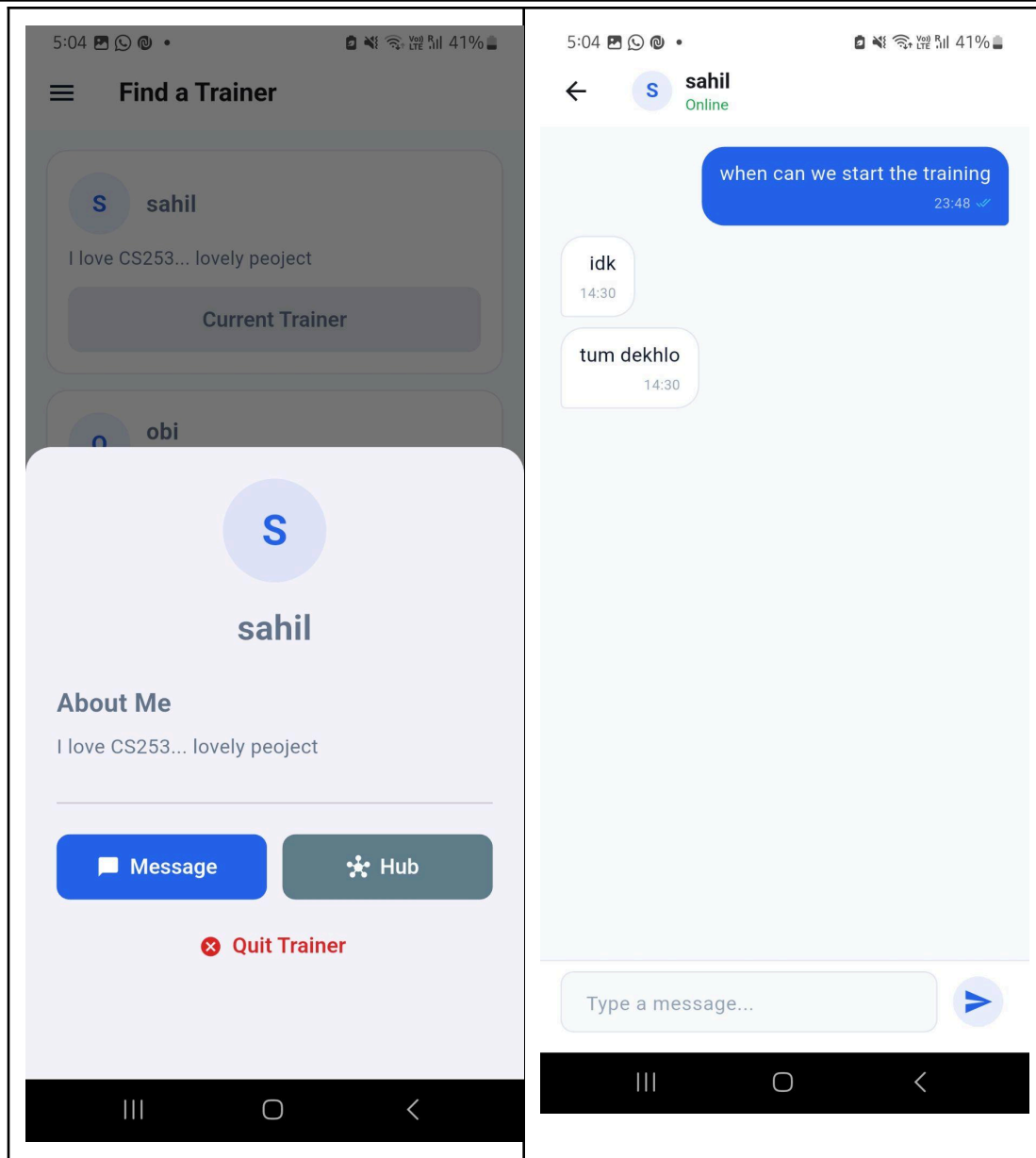
- 1.6. Social connectivity:** F6, U5 and U4 (leaderboard portion) have been implemented according to the SRS document. A comprehensive social suite is implemented, allowing users to search for friends, manage friend requests, interact with a community feed (posts and likes), and view a step-count-based leaderboard.





- 1.7. Trainer connection:** F7 and U5 have been implemented according to the SRS document. The application allows users to browse a list of trainers, select a coach, and access a dedicated "Coaching Hub". The communication pipeline is supported by a direct messaging system and notification alerts for trainer-student interactions.

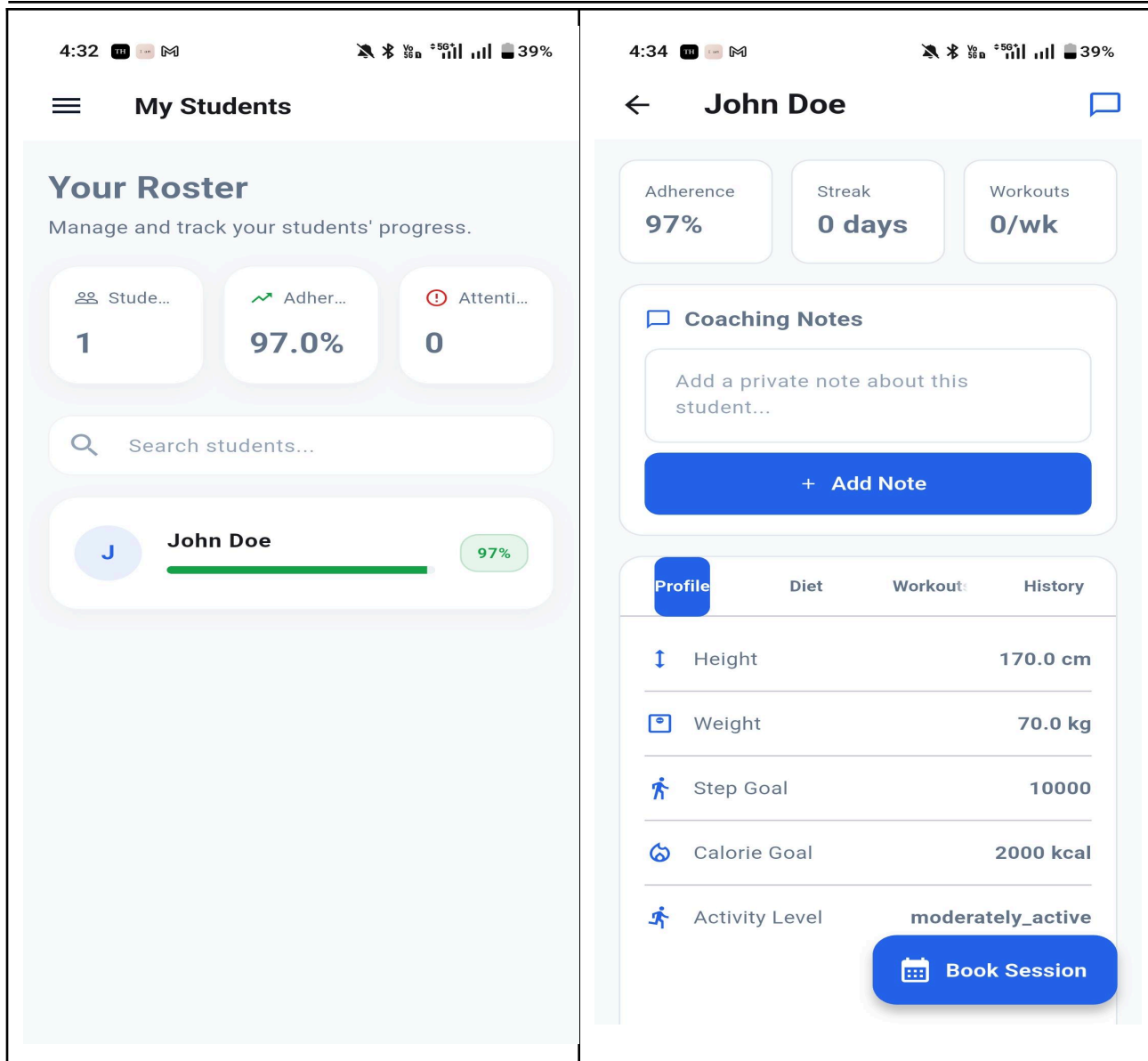


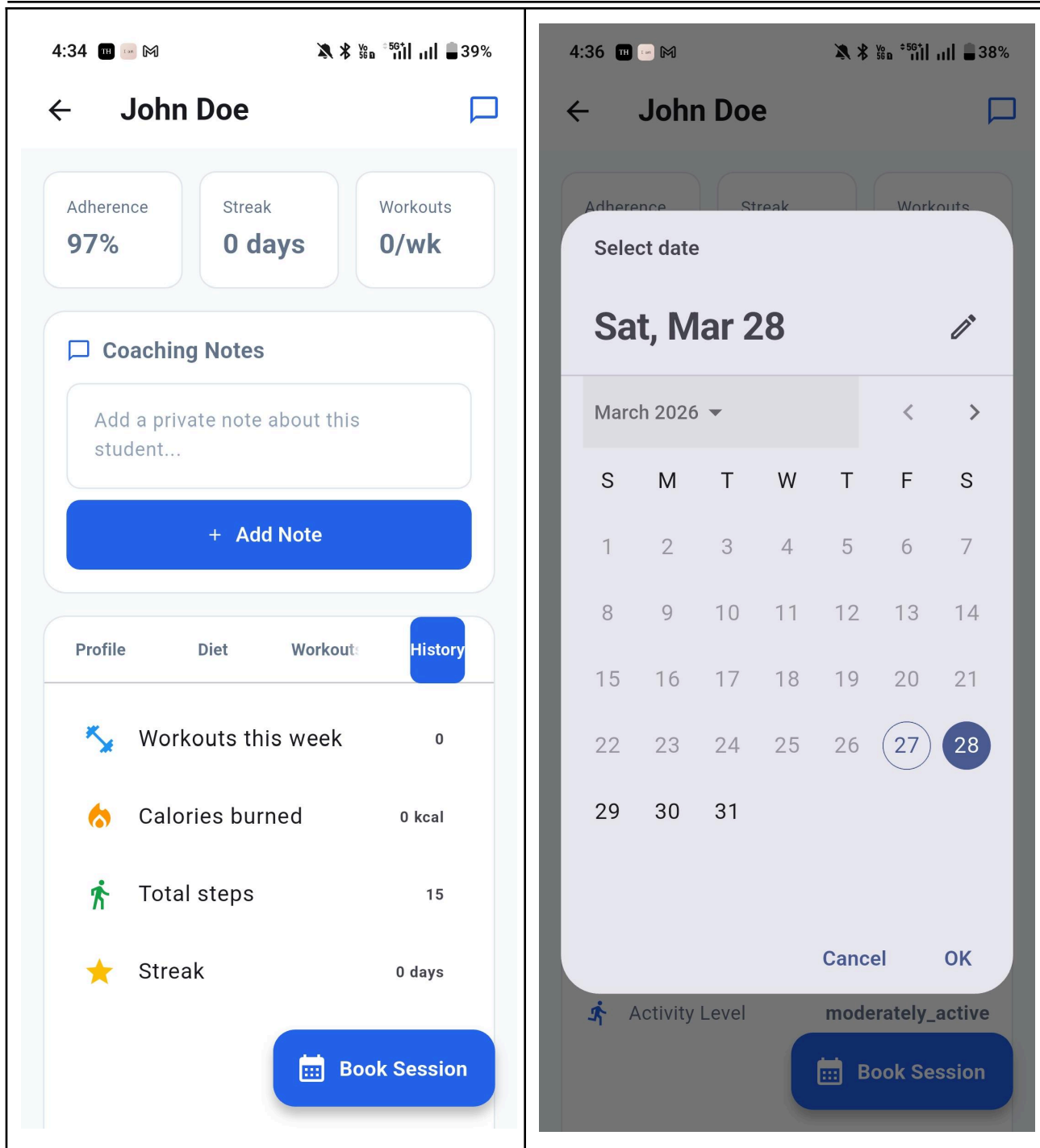


2. Implemented Features for Trainer

2.1. Student monitoring and management:

F9 and U5 have been fully implemented as defined in the SRS to support the core trainer workflow. The trainees are found in My Students. From there, the trainer is granted access to view student profile information and monitor daily progress





2.2. Request management:

F10 and U5 have been implemented according to the SRS. Trainers can review incoming trainee requests and choose to either accept or reject them. Accepting a request automatically adds the trainee to the trainer's "My Students" list.

4:33      5G+   39%

≡ Requests & Calendar

Pending Requests

1



Aditi

hdgid

✓ Accept

× Decline

Upcoming Sessions



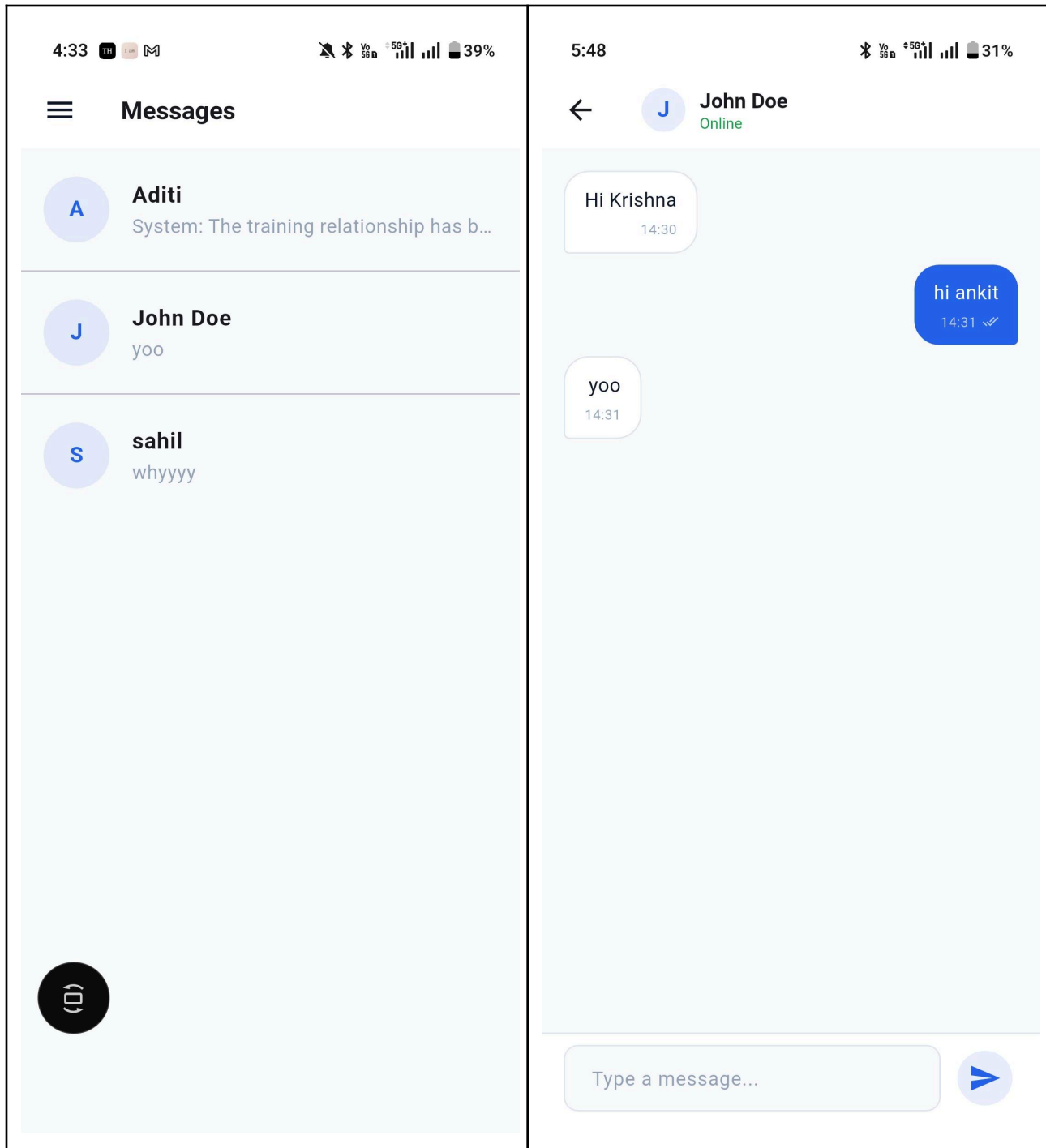
John Doe

31/3 · 10:00

60 min

2.3. Feedback and guidance:

F11 and U5 have been implemented as per the SRS. The trainer can create personalized coaching notes, send direct messages to each of their students, and schedule coaching sessions to guide them in their fitness journey.

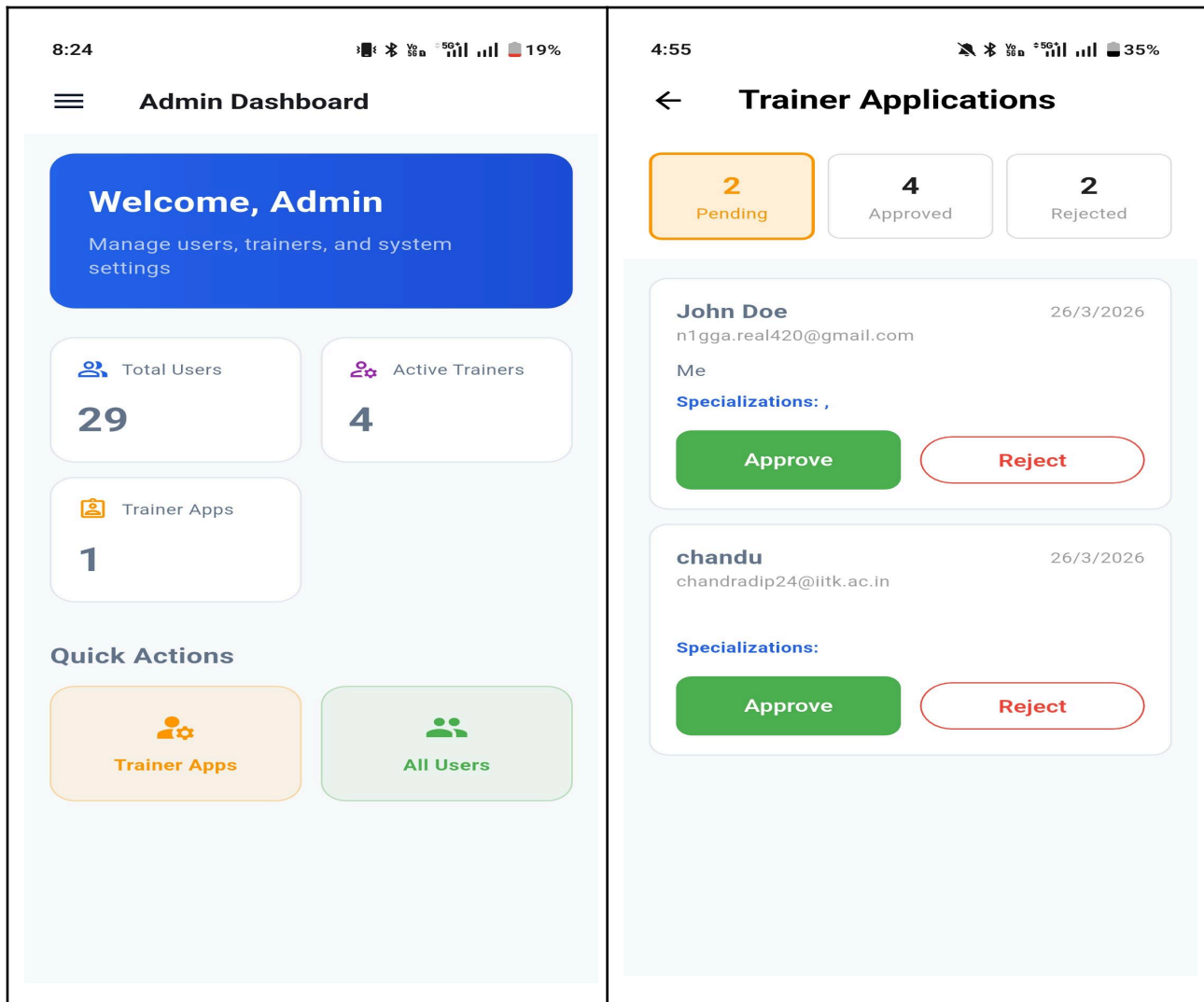


3. Implemented Features for Admin

These features are not present in the SRS and have been added additionally.

3.1. Role management

Verify professional credentials submitted by new trainers and assign trainer permissions and activate or deactivate current users



4:55

      35%

← **Trainer Applications**

2
Pending

4
Approved

2
Rejected

Krishna Agrawal

27/3/2026

krishnaa24@iitk.ac.in

hello

Specializations: jello

✕ Revoke Approval / Reject

obi

27/3/2026

opate2007@gmail.com

Me a trainee, hualalalu

Specializations: are, nhi, yaar, daamn crazy shiz, are baapre

✕ Revoke Approval / Reject

Raman

26/3/2026

ramanvatsal71@gmail.com

Specializations:

✕ Revoke Approval / Reject

4:55

      35%

← **Trainer Applications**

2
Pending

4
Approved

2
Rejected

Aditi

27/3/2026

kararwaladiti@gmail.com

nothing

Specializations: nothing

✓ Re-Approve Trainer

Matt Murdock

27/3/2026

siddiqueabubakr07@gmail.com

asd

Specializations: sd, ok

✓ Re-Approve Trainer

4:56

 35%

 **Settings**

Friend Requests

☒

Trainer Requests

☒

New Messages

☒

Post Likes

☒

System Notifications

☒

Fitness Goals

Step Goal

10000

Calorie Goal

Water Goal (ml)

2500

Activity Level

▼

Save Profile

Danger Zone

 Logout

4:56

 35%

 **Users**

U

Umar
2007umarfaruque@gmail.com
Role: trainee

☒

A

Ayush
aditikararwal29@gmail.com
Role: trainee

☒

J

John Doe
ankit.jha2807@gmail.com
Role: trainee

☒

D

David
davidfinchernine@gmail.com
Role: trainee

☐

A

Ankit
ankitkj24@iitk.ac.in
Role: trainee

☐

M

Marshall
marshallmatherska@gmail.com
Role: trainee

☐

A

ayush
ayushyra222324@gmail.com
Role: trainee

☐

Future Improvements

- **AI Summary:** We plan to introduce an LLM-based activity summary for trainers to better track their work done throughout and get insightful suggestions about how to progress their activities in the future.
- **Profile Picture:** We will allow users and trainers to upload custom avatars or photos, creating a more personalized app experience and fostering a stronger sense of connection and accountability.
- **Calendar-Based Sessions:** We aim to integrate an interactive calendar view that allows users to seamlessly schedule upcoming workouts, log future meals etc.
- **Database Expansion:** We plan to significantly broaden our existing food and exercise databases, adding regional cuisines, specialized diets (like keto or vegan), and niche workout variations to cater to a diverse user base.
- **More Accurate Calorie Tracking and Calculation:** We will refine our tracking algorithms by incorporating dynamic variables such as detailed macronutrient breakdowns to provide users with highly precise, individualized nutritional data.
- **Chat Improvement:** We intend to upgrade our messaging system to support interactive group chats, enabling users to communicate with their trainers alongside peers for shared motivation, group challenges, and community support.

Appendix A - Group Log

Date	Timings	Duration	Minutes
8th Feb 2026	3:00 pm to 4:00 pm	1 hour	Kickoff for the implementation phase. Discussed task delegation, tech stack requirements, and initial architecture planning.
15th Feb 2026	5:00 pm to 7:30 pm	2 hours 30 minutes	Deep dive into the system design. Finalized database schemas, API structures, and drafted the initial sections of the Implementation Document.
5th Mar 2026	11:00 am to 12:00 pm	1 hour	(Online Sync) Quick catch-up to review individual research on implementation tools and discuss unit testing frameworks.
11th Mar 2026	3:00 pm to 5:00 pm	2 hours	Finalized the core structure of the Implementation Document. Planned out the repository structure and branch management strategy.
13th Mar 2026	3:00 pm to 4:30 pm	1 hour 30 minutes	Repository Setup. Initialized the GitHub repository, configured local environments, and resolved environment/dependency clarifications.
15th Mar 2026	5:00 pm to 7:00 pm	2 hours	Wrote out the testing strategies in the Implementation

			Document. Assigned specific modules to team members for the upcoming coding phase.
17th Mar 2026	8:00 pm to 11:00 pm	3 hours	Final review and polish of the Implementation Document before coding begins. Prepared boilerplate code and organized project folders.
19th Mar 2026	9:00 am to 10:45 am	1 hour 45 minutes	Pre-coding sync. Ensured everyone's local branches were ready for the main commit phase starting on the 20th.
22nd Mar 2026	5:00 pm to 7:00 pm	2 hours	Reviewed the initial commits made since March 20th. Addressed early merge conflicts, began integrating the first set of modules, and started drafting unit tests for completed code.
25th Mar 2026	8:00 pm to 11:00 pm	3 hours	Core Integration & Debugging. Conducted a comprehensive merge of all feature modules. Focused on resolving logic inconsistencies between the frontend and backend and stabilizing the API endpoints.
26th Mar 2026	12:00 pm to 12:00 am	12 hours	Deployment & Infrastructure Setup. Successfully deployed the backend services to Render and provisioned the PostgreSQL database. Configured environment variables, managed CORS policies, and verified cloud

			connectivity.
27th Mar 2026	9:00 am to 9:00 pm	12 hours	End-to-End Testing & Final Release. Conducted rigorous system testing and hot-fixed critical bugs across the stack. Optimized frontend performance and generated the final production APK for distribution.